

Zest

Calcul scientifique simplifié sur infrastructure distribuée

*Executer des outils scientifiques sur infrastructure HPC/cloud
avec une seule commande*

White Paper — Version 1.0

Statut : Early Development

Akash Network, CLI Open Source

Acteurs cibles : CNES, Aix-Marseille Universite, CMA CGM, communautés scientifiques

Contents

1	Résumé Exécutif	4
1.1	Problème	4
1.2	Solution	4
1.3	Bénéfices pour Chercheurs et Entreprises	4
1.3.1	Pour les chercheurs	4
1.3.2	Pour les entreprises	4
1.4	Vers une Science Ouverte et Inclusive	5
2	Architecture Technique	6
2.1	Vue d'ensemble	6
2.2	Composants des	6
2.2.1	1. CLI Layer	6
2.2.2	2. Tool Registry	6
2.2.3	3. JobRunner Interface	7
2.2.4	4. Backend Implementations	7
2.2.5	5. Script Execution Custom	7
3	Workflow d'Execution	8
3.1	Flux d'une commande hpc run	8
4	Cas d'Usage Cibles	9
4.1	Bioinformatique	9
4.1.1	Analyse sequences (CNES, Universites)	9
4.1.2	BLAST / alignment (Biotech, Universite)	9
4.2	Geospatial	9
4.2.1	Traitement rasters DEM (CNES, CMA CGM logistics)	9
4.3	Scripts Custom	9
4.3.1	Analyse donnees proprietaires	9
5	Akash Network : Premier Backend et Strategie Multi-Cloud	10
5.1	Qu'est-ce qu'Akash Network ?	10
5.2	Pourquoi Akash Network ?	10
5.2.1	Avantages distinctifs	10
5.3	Limitations actuelles d'Akash et trajectoire	11
5.3.1	Limitations presentes	11
5.3.2	Développements planifiés Akash (roadmap 2025-2026)	11
5.3.3	Akash HomeNode - MVP : Vers une science participative décentralisée	11
5.4	Stratégie Multi-Cloud et Selection Backend	12
5.4.1	Selection Backend : Par commande	12
5.4.2	Selection Backend : Par profil utilisateur	12
5.4.3	Backends planifies	12
5.5	Cas d'usage par backend	13
5.5.1	Akash : Jobs scientifiques standards	13
5.5.2	HPC Traditionnel (Slurm) : Institutions	13
5.5.3	SecNumCloud : Donnees francaises regulées	13
6	Modele Economique	14
6.1	CLI Open-Source (Gratuit)	14
6.2	Service Manage (Payant / SaaS)	14
7	Statut et Roadmap	15
7.1	Phase 1 — MVP / POC (En cours)	15
7.2	Phase 2 — Hardening et Multi-Cloud	15
7.3	Phase 3 — Managed Layer et Multi-Cloud	15

8 Avantages Compétitifs	16
9 Impact pour les Acteurs Cibles	17
9.1 CNES (Centre National Etudes Spatiales)	17
9.2 Aix-Marseille Université	17
9.3 CMA CGM (Logistique Maritime)	17
9.4 Akash Network	17
10 Risques et Mitigation	18
11 Conclusion	19
11.1 Forces clés	19
11.2 Prochaines étapes	19
A Glossaire Technique	20
B References	20

1 Résumé Exécutif

Zest est un outil CLI open-source conçu pour simplifier l'exécution de charges de travail scientifiques sur infrastructure distribuée (Akash Network, HPC traditionnels, cloud).

L'objectif : *éliminer la complexité* liée à la containerisation, la configuration d'environnements et le transfert de fichiers.

1.1 Problème

Les chercheurs et ingénieurs exécutant des pipelines scientifiques intensifs font face à :

- Configuration manuelle de Docker / VMs
- Gestion complexe des transferts de fichiers
- Apprentissage de multiples outils (AWS CLI, Slurm, etc.)
- Administration d'infrastructure consommatrice de temps
- Dépendance à un seul fournisseur cloud

1.2 Solution

Une interface CLI unique, **tool-centric**, qui :

- Masque la complexité de l'infrastructure
- Automatise transfert, provisioning et cleanup
- Supporte outils pré-packagés (bioinformatique, géospatial) et scripts custom
- Fonctionne sur infrastructure décentralisée (Akash) ou traditionnelle
- Reste totalement open-source et extensible

1.3 Bénéfices pour Chercheurs et Entreprises

1.3.1 Pour les chercheurs

- **Accélération de la recherche** : Moins de temps sur l'infra, plus sur la science. Passer d'une idée à résultats en minutes, pas en semaines.
- **Reproductibilité améliorée** : Environnements isolés et versionnés garantissent que les mêmes résultats sont obtenus partout, par n'importe qui.
- **Collaboration facilitée** : Partager une commande `zest run` est plus simple que de documenter une infrastructure entière.
- **Accessibilité** : Aucune expertise DevOps requise. Les scientifiques restent scientifiques.
- **Coûts réduits** : Paiement à l'usage sur Akash Network = *jusqu'à 3x moins cher* qu'AWS ou GCP pour le calcul intensif.

1.3.2 Pour les entreprises

- **Time-to-market** : Déployer des pipelines ML/scientifiques sans dépendre d'équipes infra surchargées.
- **Flexibilité** : Utiliser n'importe quel backend (Akash, AWS, HPC local) avec la même commande.

- **Économies d'échelle** : Infrastructure décentralisée = coûts prévisibles et réduction des factures cloud.
- **Propriété intellectuelle protégée** : Calcul distribué sans dépendre d'un seul cloud provider.
- **Gouvernance simplifiée** : Audit et traçabilité intégrés pour conformité réglementaire.

1.4 Vers une Science Ouverte et Inclusive

Zest adresse un enjeu critique de l'équité scientifique mondiale : *l'accès au calcul de haute performance*.

- **Démocratisation du calcul** : Chercheurs dans les pays en développement peuvent accéder à infrastructure world-class sans investissement capital massif.
- **Infrastructure décentralisée** : Akash Network distribue le calcul à travers le monde, réduisant dépendances géopolitiques et coûts locaux.
- **Science reproductible** : Code open-source + CLI publique = transparence totale. Chacun peut vérifier, reproduire, améliorer.
- **Communauté mondiale** : Créer liens entre institutions riches et laboratoires ressources limitées. Partage de pipelines scientifiques sans frictions.
- **Impact durable** : Outils gratuits et accessibles = science plus inclusive, résultats plus robustes (peer review global).

CLI Gratuite + Service Managé Payant La CLI est open-source et gratuite. Un service web optionnel proposera gestion des jobs, historique, multi-utilisateurs et intégration pour les institutions.

2 Architecture Technique

2.1 Vue d'ensemble

HPC Flow fonctionne selon une architecture simple en trois couches :

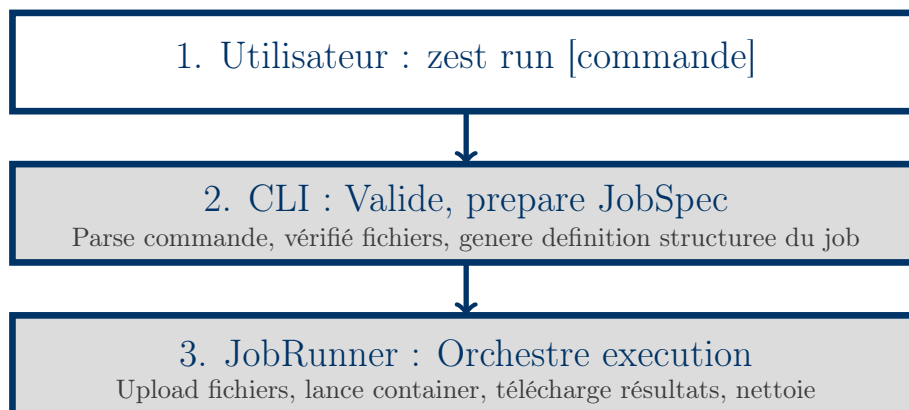


Figure 1: Flux d'exécution simplifié

2.2 Composants cles

2.2.1 1. CLI Layer

Interface utilisateur minimaliste. L'utilisateur lance une commande simple :

```
zest run vsearch \  
--input reads.fasta \  
--id 0.97 \  
--output centroids.fasta \  
--cloud akash
```

La CLI :

- Parse les arguments
- Vérifie que le fichier `reads.fasta` existe localement
- Consulte le Tool Registry pour trouver `vsearch`
- Construit une spécification de job (JobSpec) en format structurée

Pas de Docker, pas de VM, pas de connaissance infrastructure requise.

2.2.2 2. Tool Registry

Catalogue YAML de outils pre-packages. Exemple pour `vsearch` :

```
name: vsearch  
version: "2.28.1"  
image: quay.io/biocontainers/vsearch:2.28.1  
parameters:  
- name: input  
  flag: --fastx_uniques  
- name: id  
  flag: --id  
resources:
```

```
cpu: 4
memory: 8Gi
```

Chacun peut contribuer a de nouveaux outils (bioinformatique, geospatial, etc.).

2.2.3 3. JobRunner Interface

Abstraction générique permettant de soutenir différents backends sans changer le code CLI.

```
interface JobRunner:
- submit(job) -> job_id
- status(job_id) -> pending|running|succeeded|failed
- fetch_results(job_id) -> fichiers_locaux
- cleanup(job_id)
```

2.2.4 4. Backend Implementations

Phase 1 (MVP) : AkashJobRunner

- Upload fichiers d'entrée sur Akash
- Crée un deployment Akash avec l'image du tool
- Monitore l'exécution via logs Akash
- Télécharge les résultats
- Detruit le deployment

Phase 2+ : Support AWS, GCP, OVH, Slurm

- AWSJobRunner : utilise Lambda ou EC2
- GCPJobRunner : utilise Cloud Run ou Compute Engine
- SlurmJobRunner : intégration HPC traditionnels

2.2.5 5. Script Execution Custom

Pour les scientifiques avec pipelines spécifiques :

```
zest run-script mon_analyse.py \
--runtime python:3.11 \
--input data.csv \
--output result.html \
--cloud akash
```

Le script et ses dépendances (`requirements.txt`) sont uploadés et exécutés dans un runtime de base.

3 Workflow d'Execution

3.1 Flux d'une commande `hpc run`

1. **Parsing** : CLI interprète la commande et flags
 2. **Validation** : Controle existence fichiers, valide flags, ressources disponibles
 3. **JobSpec construction** : Creation d'une description structurée du job
 4. **Backend selection** : Choix du JobRunner (Akash, AWS, etc.)
 5. **Resource provisioning** : Allocation ressources sur le backend
 6. **File upload** : Transfert fichiers d'entrée vers remote
 7. **Job execution** : Lancement du container avec arguments
 8. **Monitoring** : Polling status, capture logs
 9. **Result download** : Récupération fichiers output en local
 10. **Cleanup** : Destruction du deployment
- Durée totale** : Quelques secondes (provisioning) + temps calcul + quelques secondes (download).

4 Cas d'Usage Cibles

4.1 Bioinformatique

4.1.1 Analyse sequences (CNES, Universites)

Chercheurs en biologie evolutive necessitent clustering de milliers de sequences ADN.

```
zest run vsearch \  
--input metagenome_1M_reads.fasta \  
--id 0.97 \  
--threads 16 \  
--output centroids.fasta \  
--cloud akash
```

Avantage : Une commande. Pas d'administration infrastructure.

4.1.2 BLAST / alignment (Biotech, Universite)

```
zest run blast \  
--query proteins.fasta \  
--db nr \  
--output results.tsv \  
--cloud akash
```

4.2 Geospatial

4.2.1 Traitement rasters DEM (CNES, CMA CGM logistics)

Calcul de pentes, d'exposition, d'indices de vegetation sur images satellites.

```
zest run geoprocess \  
--input dem_region.tif \  
--operation slope \  
--output slope_map.tif \  
--cloud akash
```

Cas d'usage CMA CGM : Optimisation logistique via analyse relief terrain pour planification trajets.

4.3 Scripts Custom

4.3.1 Analyse donnees proprietaires

Chercheur avec pipeline specifique (combinaison outils maison + libs standard).

```
zest run-script analyse_propriete.py \  
--runtime python:3.11 \  
--input data_confidentiel.csv \  
--output rapport.html \  
--cloud akash
```

Fichier requirements.txt est téléchargé automatiquement.

5 Akash Network : Premier Backend et Strategie Multi-Cloud

5.1 Qu'est-ce qu'Akash Network ?

Akash Network est une marketplace décentralisée de ressources cloud basée sur la technologie blockchain. Contrairement aux fournisseurs cloud traditionnels (AWS, Google Cloud, Azure), Akash fonctionne en peer-to-peer : les utilisateurs achètent et vendent de la puissance de calcul (CPU, GPU, mémoire) directement auprès de fournisseurs indépendants (nodes), sans intermédiaire centralisé.

Fonctionnement clé :

Les fournisseurs (nodes) proposent leurs ressources inutilisées sur le réseau. Les utilisateurs paient avec des tokens AKASH (AKT) pour louer ces ressources (Le but du service managé est d'éliminer cette étape pour l'utilisateur). Les deployments sont isolés et sécurisés grâce à des conteneurs Kubernetes natifs. Coûts réduits : Jusqu'à 80% moins chers que les clouds centralisés grâce à l'absence d'intermédiaires. Infrastructure ouverte : Pas de lock-in, compatibilité avec les outils standards (Docker, Kubernetes). Décentralisé : Pas de dépendance à un seul fournisseur, résilience accrue.

Akash est souvent surnommé le "Airbnb du cloud" car il permet aux propriétaires de matériel de monétiser leurs ressources inutilisées, tout en offrant aux utilisateurs une alternative économique et flexible aux clouds traditionnels.

5.2 Pourquoi Akash Network ?

Akash Network est choisi comme **premier backend** pour Zest pour plusieurs raisons strategiques :

5.2.1 Avantages distinctifs

Critere	Akash	AWS	GCP
Décentralisé	✓	×	×
Prix 60-80% moins cher	✓	Baseline	≈ AWS
Pas de lock-in fournisseur	✓	×	×
Marché ouvert (peer-to-peer)	✓	×	×
Facile intégration containers	✓	✓	✓

Table 1: Comparaison Akash vs cloud centralisé

1. Décentralisation et Souveraineté Akash est une blockchain décentralisée. Aucun acteur unique ne controle l'infrastructure. Idéal pour :

- CNES : pas de dépendance geopolitique pour donnees spatiales
- CMA CGM : données logistiques sensibles non centralisées
- Universités : data governance flexible

2. Economies d'échelle Prix **60-80% moins chers** que AWS/GCP grâce au modele P2P. Les fournisseurs (providers) Akash optimisent leurs coûts.

3. Absence de lock-in Architecture multi-backend permet switch vers autre cloud sans changer de code utilisateur. Levier concurrentiel durable.

4. Marche ouvert Providers multiples (node operators) enchérissent pour jobs. Transparence des prix. Pas de tarification captive.

5.3 Limitations actuelles d'Akash et trajectoire

5.3.1 Limitations presentes

- **Maturité réseau** : Variabilité de disponibilité compute. Certains providers peuvent être offline.
- **Offre limitée compute** : Moins de GPU haute performance que AWS. Mieux adapté a CPU/GPU standard.
- **Stockage objet** : Pas d'équivalent S3 natif. Fichiers volumineux requièrent up/download.
- **Ecosysteme outillage** : Outils monitoring et observabilité moins matures que AWS CloudWatch.
- **Support technique** : Communauté émergente, moins de support entreprise.
- **SLA garanties** : Pas de garanties SLA formelles (work in progress).

5.3.2 Développements planifiés Akash (roadmap 2025-2026)

Akash Network poursuit son évolution avec plusieurs améliorations majeures prévues :

- **GPU Support amélioré** : Support avancé pour les GPU NVIDIA H100, A100 et H200 avec optimisation des performances pour l'IA/ML et le calcul intensif
- **Amélioration des garanties SLA** : Mise en place de garanties SLA formelles et automatisées pour une meilleure fiabilité des déploiements
- **Solution de stockage décentralisée** : Déploiement d'une solution de stockage compatible S3 avec intégration native à l'écosystème Akash
- **Écosystème des fournisseurs** : Expansion significative du nombre de providers avec des mécanismes de qualité de service renforcés et un système de réputation amélioré
- **Outils de monitoring et observabilité** : Tableau de bord unifié et outils avancés pour le monitoring des déploiements et des providers
- **Optimisation des coûts** : Réduction continue des coûts de déploiement avec des mécanismes de tarification dynamique

5.3.3 Akash HomeNode - MVP : Vers une science participative décentralisée

Akash Network a annoncé le développement d'**Akash HomeNode**, une solution en version MVP (Minimum Viable Product) visant à démocratiser l'accès aux ressources informatiques décentralisées. Cette initiative représente une opportunité majeure pour la **science participative** en permettant :

- **Calcul distribué citoyen** : Transformation des ordinateurs domestiques en nœuds de calcul contribuant à des projets scientifiques (recherche médicale, astronomie, climatologie, etc.)
- **Infrastructure accessible** : Réduction des barrières à l'entrée pour les chercheurs et citoyens souhaitant participer à des initiatives de calcul distribué
- **Applications concrètes** :
 - Analyse de données biomédicales
 - Modélisation climatique
 - Recherche sur les maladies rares
 - Traitement de données astronomiques

Cette approche s'inscrit dans la philosophie d'Akash Network de créer une infrastructure cloud véritablement **démocratique et décentralisée**, où chaque individu peut contribuer à l'avancement scientifique tout en bénéficiant d'une rémunération pour ses ressources partagées.

5.4 Stratégie Multi-Cloud et Selection Backend

Vision long-terme : Zest abstrait l'infrastructure. L'utilisateur choisit backend selon besoin.

5.4.1 Selection Backend : Par commande

L'utilisateur spécifie backend directement dans la commande :

```
# Utiliser Akash (par default, economique)
zest run vsearch --input data.fasta --cloud akash

# Utiliser AWS pour job critique necessite SLA 99.99%
zest run blast --input proteins.fasta --cloud aws --
    instance-type c5.4xlarge

# Utiliser HPC traditionnel pour GPU haute perf
zest run deeplearning --input train.tar --cloud slurm --
    partition gpu
```

5.4.2 Selection Backend : Par profil utilisateur

Phase 2 : Configuration persistante dans ~/.hpc/config.yaml :

```
# Profil par default
default_cloud: akash

# Overrides par type de job
cloud_rules:
- match:
  tool: blast
  input_size: ">10GB"
  cloud: aws

- match:
  tool: deeplearning
  cloud: aws
  gpu: true
  instance_type: p3.8xlarge

- match:
  runtime: python
  memory_required: ">16Gi"
  cloud: slurm
  partition: gpu
```

Avantages :

- **Flexibilité** : Scientifique choisit le meilleur cloud par cas d'usage
- **Optimisation couts** : Akash par défaut pour jobs standards, OVH ou autres acteurs centralisés pour critiques
- **Pas de re-apprentissage** : Syntaxe CLI reste identique

5.4.3 Backends planifiés

Phase 1 (2024-Q2) :

- AkashJobRunner (MVP)

Phase 2 (2024-Q4) :

- AWSJobRunner (EC2 + Lambda)
- Local Slurm JobRunner (HPC traditionnel)

Phase 3 (2025-Q1+) :

- GCPJobRunner (Compute Engine + Cloud Run)
- SecNumCloud JobRunner (infrastructure française souveraine)
- Azure JobRunner
- Kubernetes generic JobRunner (on-prem, managed k8s)

5.5 Cas d'usage par backend

5.5.1 Akash : Jobs scientifiques standards

Ideal pour :

- Jobs CPU/GPU standards (non-critique)
- Workloads bioinformatique (vsearch, BLAST, SAMtools)
- Geospatial processing avec GDAL/QGIS
- Recherche académique (budget constraints)
- Développement et expérimentation (prototypage)

Prix estimatif (approximations 2026) :

- CPU 4-core, 8GB RAM : \$0.05-0.10 par heure vs \$0.25 AWS
- GPU L40 : \$0.30-0.50 par heure vs \$1.00+ AWS

5.5.2 HPC Traditionnel (Slurm) : Institutions

Ideal pour :

- Universités/Labos avec infrastructure interne
- Workloads MPI parallèles massifs
- Données sensibles on-premise
- Intégration pipelines HPC existants

5.5.3 SecNumCloud : Données françaises régulées

Ideal pour :

- Entités publiques : Conformité cloud souverain français
- Biotech pharma : Données patients confidentielles
- Infrastructure 100% France hébergement garantis

6 Modele Economique

6.1 CLI Open-Source (Gratuit)

- Code source MIT licence
- Installation locale : `pip install hpc-flow`
- Utilisation illimitée (couts infra a charge utilisateur)
- Contributions de la communauté bienvenues

6.2 Service Manage (Payant / SaaS)

Phase 2-3 : plateforme web optionnelle pour :

- **Gestion multi-utilisateurs** : Teams, projets, quotas
- **Historique et monitoring** : Jobs, logs, durée, couts
- **Dashboard web** : Lancement jobs, suivi, resultats
- **Integration API** : Appels programmatiques
- **Support technique** : SLA garanti

Tarification estimée (Phase 3) Modele freemium : gratuit jusqu'a X jobs/mois, puis tarification a l'usage ou abonnement institutionnel.
--

7 Statut et Roadmap

7.1 Phase 1 — MVP / POC (En cours)

Objectif : CLI fonctionnelle + backend Akash + 2-3 outils de demo.

- ✓ Architecture générique JobRunner
- ✓ Implémentation AkashJobRunner
- ✓ CLI parsing et JobSpec
- ✓ Tool registry (vsearch, geoprocess)
- ✓ File transfer (upload/download)
- ✓ Script execution (Python)
- Démonstration POC (bioinformatique + géospatial)

Timeline : 2-3 mois

7.2 Phase 2 — Hardening et Multi-Cloud

Objectif : CLI production-ready, support AWS + Slurm, configuration profile.

- Outils additionnels (BLAST, SAMtools, GDAL complets)
- Runtimes script : Bash, R, Julia
- Job history local (zest log, zest history)
- AWSJobRunner implémentation
- SlurmJobRunner implémentation
- Config file support (~/.hpc/config.yaml)
- Cloud selection logic
- Error handling robuste

Timeline : 3-4 mois après Phase 1

7.3 Phase 3 — Managed Layer et Multi-Cloud

Objectif : SaaS optionnel, support multi-cloud (GCP, SecNumCloud, Azure).

- Web app / dashboard
- User et project management
- GCPJobRunner implementation
- SecNumCloud intégration (partnership)
- Azure support
- API REST
- Pricing engine (cost estimation)
- Advanced resource prediction

Timeline : 4-6 mois après Phase 2

8 Avantages Compétitifs

Critère	zest	Cloud classique	HPC trad.
Pas Docker/VM requis	✓	×	×
Interface tool-centric	✓	×	×
Decentralise / multi-cloud	✓	×	×
Transfert fichiers auto	✓	Manuel	Manuel
Cleanup automatique	✓	Manuel	Manuel
Open-source et extensible	✓	×	Partiel
Economique (pay-as-you-go)	✓	✓	Abonnement
Courbe apprentissage faible	✓	×	×

Table 2: Positionnement marche

9 Impact pour les Acteurs Cibles

9.1 CNES (Centre National Etudes Spatiales)

- **Défi** : Traitement massif de données géospatiales (satellites), infrastructure coûteuse, souveraineté données
- **Apport zest** : Exécution pipelines géospatial sur Akash ou SecNumCloud (économies + souveraineté), API extensible, pas de lock-in fournisseur
- **Cas d'usage** : Analyse DEM, détection objets, classification satellite
- **Gain économique** : -60% couts compute vs AWS avec Akash

9.2 Aix-Marseille Université

- **Defi** : Chercheurs hétérogènes, pas d'équipe infra dédiée, couts cloud élevés
- **Apport HPC Flow** : Outil simple pour tous, auto-hébergé ou sur Akash, pas d'administration
- **Cas d'usage** : Bioinformatique, chimie quantique, géospatial, analyses data science
- **Gain** : Démocratisation accès HPC, réduction charge IT

9.3 CMA CGM (Logistique Maritime)

- **Defi** : Optimisation trajets, prédictions, besoin compute on-demand sans over-capacity
- **Apport zest** : Intégration facile dans pipelines data, scaling élastique, coûts proportionnels, choix cloud (Akash ou intégration interne)
- **Cas d'usage** : Analyse relief (trajets marins), prédictions météo, optimisation routes
- **Gain** : Flexibilité coûts, pas d'over-provisioning

9.4 Akash Network

- **Defi** : Augmenter adoption compute décentralisé, proposer outils hauts niveaux
- **Apport HPC Flow** : Killer app pour scientifiques, abstraire complexité Akash, bootstrap communauté scientifique
- **Synergies** : Co-marketing, intégration Akash API, case studies communs, accélération providers adoption

10 Risques et Mitigation

Risque	Description	Mitigation
Akash Network immaturité	Variabilité prix, disponibilité compute	Multi-cloud.
Adoption lente	Chercheurs habitués aux outils existants	POC public + démos, partenariats institution et entreprise
Fragmentation registry	Trop d'outils, mauvaise maintenance	Governance claire, guidelines, CI/CD, code review
Scaling limite Akash	Bottleneck transfert fichiers volumineux	S3 URLs, streaming, multi-cloud fallback
Support utilisateurs	Demande support technique croissante	SaaS managed Phase 3, docs excellentes, communauté
Compatibilité backends	Divergences syntaxe AWS vs GCP	Abstraction uniforme JobRunner

11 Conclusion

HPC Flow adresse un besoin réel et universel : simplifier l'exécution de charges de travail scientifique sans expertise infra, tout en préservant la **liberté de choix** sur l'infrastructure.

11.1 Forces clés

1. **Simplicité** : Une commande pour tout. Pas Docker, pas VM, pas de tracas.
2. **Flexibilité** : Open-source, extensible, multi-cloud par architecture.
3. **Décentralisation** : Akash Network comme premier backend, autres possibles.
4. **Economies** : 60-80% réduction coûts compute vs cloud centralisé (Akash Phase 1).
5. **Communauté** : Registry contributif, MIT licence, contributions bienvenues.
6. **Modele durable** : CLI gratuite + SaaS optionnel.
7. **Multi-cloud** : Jamais enfermé. Eviter lock-in fournisseur.

11.2 Prochaines étapes

- Phase 1 : POC fonctionnel Akash (fin Q2 2024)
- Partenariats : CNES, Aix-Marseille, Akash Network
- Publication + présentation conférences scientifiques (JRES, ADP)
- Recrutement contributeurs open-source
- Phase 2 : Multi-cloud (AWS, Slurm) fin Q4 2024

HPC Flow : rendre le calcul scientifique accessible a tous, avec choix infrastructure et coûts optimisés.

A Glossaire Technique

Terme	Définition
HPC	High Performance Computing - Calcul haute performance
Akash Network	Marché décentralisé peer-to-peer de ressources computationnelles blockchain
CLI	Command-Line Interface - Interface en ligne de commande
JobSpec	Spécification structurée d'un job (outil, args, fichiers, ressources)
JobRunner	Abstraction pour soumettre jobs à un backend (Akash, AWS, Slurm)
Registry	Catalogue de définitions d'outils
POC	Proof of Concept - Validation technique d'un concept
YAML	Format texte lisible pour données structurées
API	Application Programming Interface - Interface programmation
SLA	Service Level Agreement - Accord niveau service
P2P	Peer-to-Peer - Architecture distribuée sans intermédiaire central
Lock-in	Dépendance à un seul fournisseur sans moyen d'en changer
SecNumCloud	Référentiel français cloud souverain, sécurité renforcée

B References

-
- GitHub Repository : <https://github.com/hdavid-13/hpc-flow>
 - Akash Network : <https://akash.network>
 - Akash Roadmap : <https://github.com/akash-network/roadmap>
 - Biocontainers : <https://biocontainers.pro>
 - SecNumCloud : <https://www.ssi.gouv.fr/entreprise/qualifications/>